

به نام خدا

گزارش کار پروژه‌ی مهندسی نرم افزار

دکتر صمدی -- بهار ۱۴۰۵

عنوان پروژه: سامانه خرید و فروش دیجیتال با امکان مدیریت لایسنس

Digital Marketplace Platform

اعضای تیم:

- ماهان بانسی -- ۴۰۲۲۴۳۰۴۲
- روزبه سلطانی -- ۴۰۲۲۴۳۰۷۲
- سیدمحمد مهدی میرمطهری -- ۴۰۲۲۴۳۱۰۶
- عرفان پنجه شاهی -- ۴۰۲۲۴۳۰۴۶

مقدمه

هدف این پروژه، طراحی و پیاده‌سازی یک سامانه‌ی خرید و فروش دیجیتال (Digital Marketplace) است که در ترم بهار ۱۴۰۵ در دانشگاه شهید بهشتی، در قالب پروژه‌ی درس مهندسی نرم‌افزار و تحت نظر دکتر صمدی انجام شده است. این سامانه بستری را فراهم می‌کند تا فروشندگان بتوانند محصولات دیجیتال مانند دوره‌های آموزشی، کتاب‌های الکترونیکی و لایسنس‌های نرم‌افزاری را عرضه کنند و خریداران بتوانند این محصولات را جست‌وجو، خریداری و مدیریت کنند. سامانه‌ی نهایی امکانات زیر را در اختیار کاربران قرار می‌دهد:

- ثبت‌نام و ورود امن کاربران با استفاده از سیستم احراز هویت مبتنی بر JWT و رمزنگاری کلمه عبور با bcrypt.
- ثبت، ویرایش و مدیریت محصولات دیجیتال توسط فروشنده، با کنترل مالکیت محصول.
- مشاهده‌ی محصولات، افزودن به سبد خرید و تسویه‌حساب توسط خریدار، و دسترسی به محصولات خریداری‌شده در داشبورد شخصی.
- مشاهده‌ی مشتریان، سفارش‌ها و آمار و تحلیل‌های فروش (Seller Analytics) توسط فروشنده.
- تأیید یا رد محصولات، مدیریت کاربران (بن کردن، تغییر نقش) و نظارت کلی بر پلتفرم توسط ادمین.
- مدیریت پلن‌های اشتراک (Subscription) و دسترسی خریداران دارای اشتراک فعال به محتوای اختصاصی.
- از نظر معماری، پروژه به دو بخش کاملاً مجزا تقسیم شده که از طریق یک REST API با یکدیگر ارتباط برقرار می‌کنند:
- سمت فرانت‌اند (Front-end) با استفاده از React و Vite پیاده‌سازی شده و برای استایل‌دهی و طراحی واکنش‌گرا (Responsive) از Tailwind CSS بهره می‌برد.
- سمت بک‌اند (Back-end) با Node.js و فریم‌ورک Express پیاده‌سازی شده و داده‌ها را در پایگاه‌داده‌ی PostgreSQL ذخیره می‌کند که از طریق Prisma ORM مدیریت می‌شود.
- علاوه بر پیاده‌سازی، در طول پروژه به مستندسازی معماری (با مدل C4)، نوشتن تست‌های واحد برای سرویس‌های بک‌اند با Jest و Supertest، داکرایز کردن کامل پروژه با Docker و Docker Compose، و راه‌اندازی فرآیند یکپارچه‌سازی مداوم (CI) با GitHub Actions نیز پرداخته شده است. در ادامه‌ی این گزارش، فعالیت‌های انجام‌شده در هر یک از سه فاز پروژه به تفکیک شرح داده می‌شود.

۱. فاز اول پروژه: شناخت و تحلیل نیازمندی‌ها

در این فاز، تمرکز اصلی تیم بر درک مسئله، شناخت کاربران و استخراج نیازمندی‌های سیستم بود. هدف از این فاز آن بود که پیش از شروع پیاده‌سازی، دقیقاً مشخص شود چه سیستمی قرار است ساخته شود و چرا. برای رسیدن به این هدف، سه سند کلیدی زیر تهیه شد که هر کدام بخشی از تصویر کلی پروژه را کامل می‌کنند:

۱.۱ سند چشم‌انداز (Vision)

این سند تصویر کلی و جهت‌گیری پروژه را مشخص می‌کند. در واقع توضیح می‌دهد که پروژه چه مشکلی را حل می‌کند و چرا اصلاً ارزش انجام دادن دارد. در این سند مسئله‌ی اصلی -- نبود بستری یکپارچه برای عرضه و فروش امن محصولات دیجیتال همراه با مدیریت لایسنس -- تشریح شد و ارزش پیشنهادی سامانه برای دو گروه اصلی کاربران، یعنی فروشندگان و خریداران محصولات دیجیتال، بیان گردید.

۱.۲ سند نیازمندی‌ها (SRS)

در این سند وارد جزئیات شدیم و مشخص کردیم سیستم دقیقاً چه کارهایی باید انجام دهد. مطابق با ساختار خواسته‌شده، نیازمندی‌ها به دو دسته‌ی عملکردی (کارهایی که سیستم انجام می‌دهد، مانند ثبت‌نام، ثبت محصول، خرید و تسویه‌حساب) و غیرعملکردی (ویژگی‌هایی مانند سرعت پاسخ‌دهی، امنیت احراز هویت، و قابلیت استفاده‌ی رابط کاربری) تفکیک شدند. برای بخش‌های مهم سامانه، سناریوهای کاربردی (Use Case) نظیر «ثبت‌نام کاربر»، «ثبت محصول توسط فروشنده»، «خرید محصول توسط خریدار» و «تأیید محصول توسط ادمین» تعریف شد و برای نمایش رفتار و تعاملات سیستم، نمودارهای Activity و Sequence مربوط به این سناریوها رسم گردید. هدف از این سند آن بود که هیچ ابهامی درباره‌ی «سیستم چه باید بکند» باقی نماند؛ نیازمندی‌های عملکردی نهایی‌شده در این سند، مبنای پیاده‌سازی در فازهای بعدی و همچنین مبنای جدول تطابق نیازمندی‌ها در فاز سوم قرار گرفتند.

۱.۳ سند تحلیل ریسک (Risk Analysis)

در این سند ریسک‌های احتمالی پروژه پیش از وقوع شناسایی شدند تا بتوان برای آن‌ها برنامه داشت. ریسک‌هایی نظیر مشکلات فنی در یکپارچه‌سازی فرانت‌اند و بک‌اند، تأخیر زمانی ناشی از هماهنگی تیمی، و کمبود تجربه در برخی فناوری‌های جدید (مانند Prisma ORM) شناسایی شدند، احتمال وقوع و شدت تأثیر هر کدام بررسی شد، ماتریس ارزیابی ریسک ارائه گردید و برای کاهش یا مدیریت هر ریسک، راهکاری مشخص تعریف شد. هدف این بود که پروژه غافلگیر نشود و تیم بتواند کنترل بهتری روی شرایط پیش رو داشته باشد.

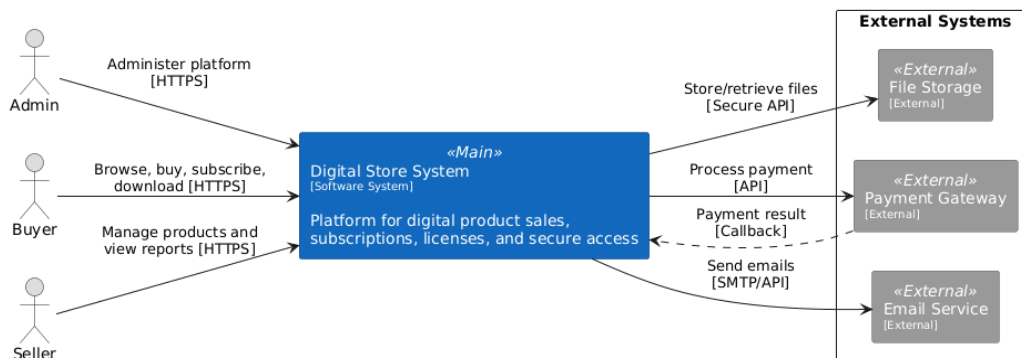
۲. فاز دوم پروژه: طراحی معماری و آغاز توسعه

در این فاز، تمرکز بر طراحی معماری سیستم، شبیه‌سازی رفتار کاربر در قالب یک نمونه‌ی اولیه (Prototype)، و آغاز پیاده‌سازی زیرساخت فنی بود. مطابق با الزامات این فاز، معماری Back-end و Front-end به‌طور کامل از یکدیگر مجزا نگه داشته شد و ارتباط میان آن‌ها صرفاً از طریق یک REST API برقرار می‌شود.

۲.۱ طراحی معماری سیستم با مدل C۴

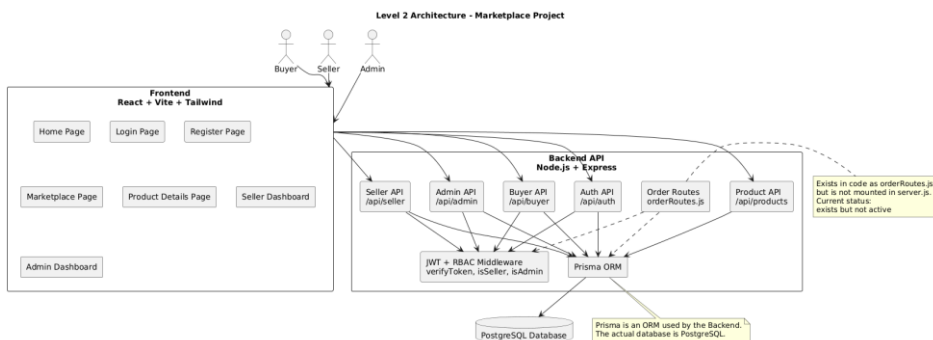
بر اساس نیازمندی‌های مستندشده در فاز اول، معماری سیستم با استفاده از مدل C۴ در سه سطح اول ترسیم شد:

- سطح ۱ -- دیگرام بافت سیستم (System Context): تعامل سیستم با کاربران (خریدار، فروشنده، ادمین) و سیستم‌های خارجی.



شکل ۱ -- دیگرام بافت سیستم (System Context) -- Level ۱

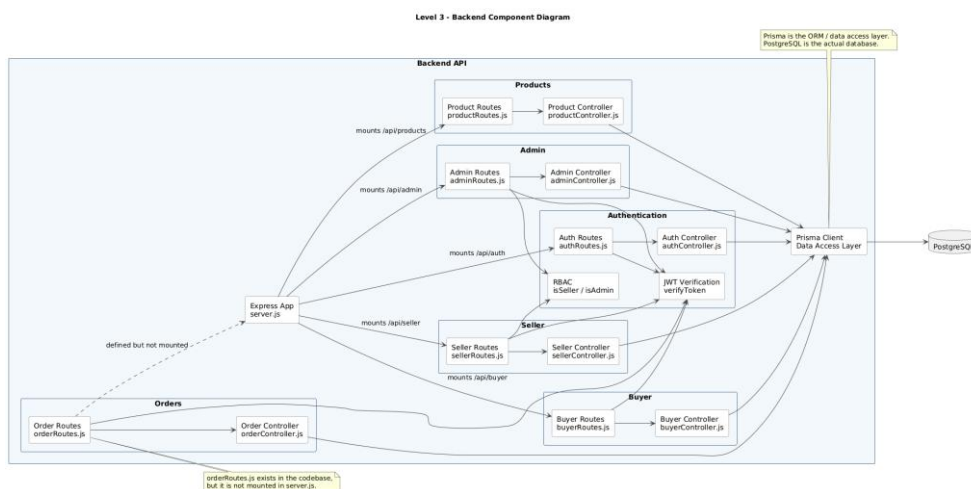
- سطح ۲ -- دیگرام کانتینرها (Container): سرویس‌ها، برنامه‌های کاربردی، پایگاه داده و ارتباط میان آن‌ها.



شکل ۲ -- دیگرام کانتینرها (Container Diagram) -- Level ۲

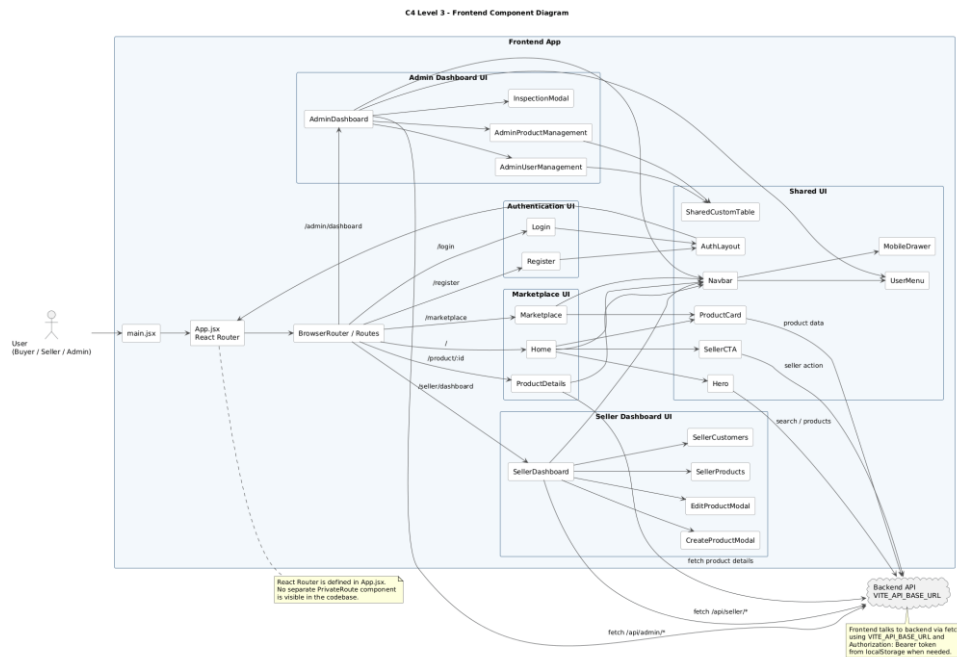
- سطح ۳ -- دیگرام کامپوننت‌ها (Component): اجزای داخلی هر کانتینر به تفکیک بک‌اند، فرانت‌اند و پایگاه داده.

دیگرام کامپوننت‌های بک‌اند:



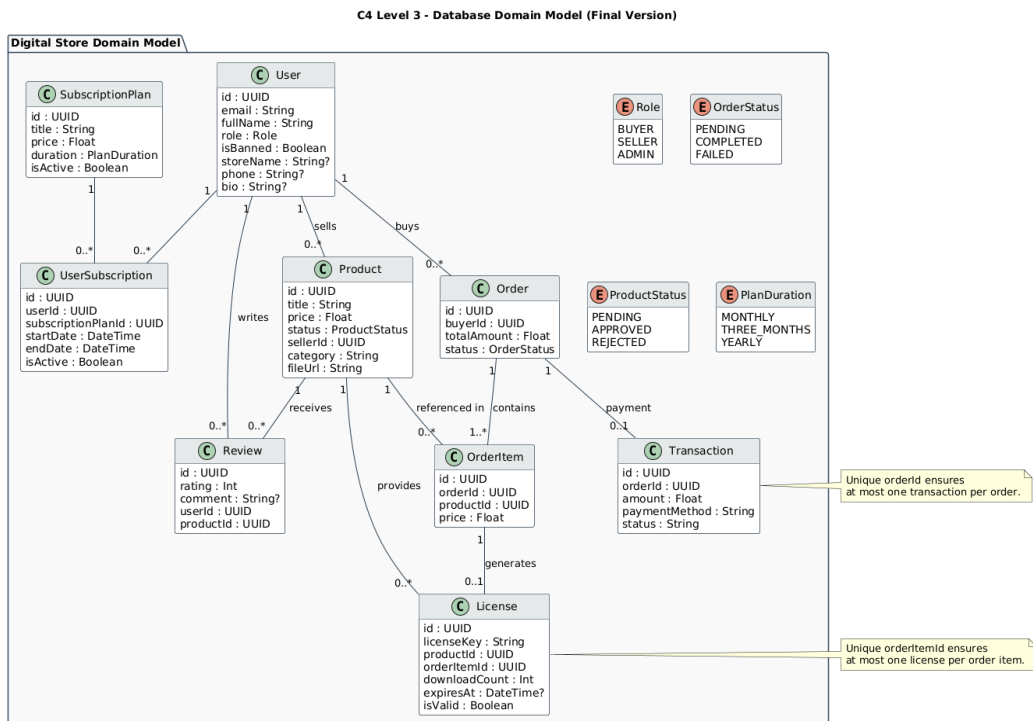
شکل ۳ -- دیگرام کامپوننت‌های بک‌اند (Backend) -- Level ۳

دیگرام کامپونت‌های فرانت‌اند:



شکل ۴ -- دیگرام کامپونت‌های فرانت‌اند (Level ۳ -- Frontend)

دیگرام مدل داده‌ی پایگاه‌داده:

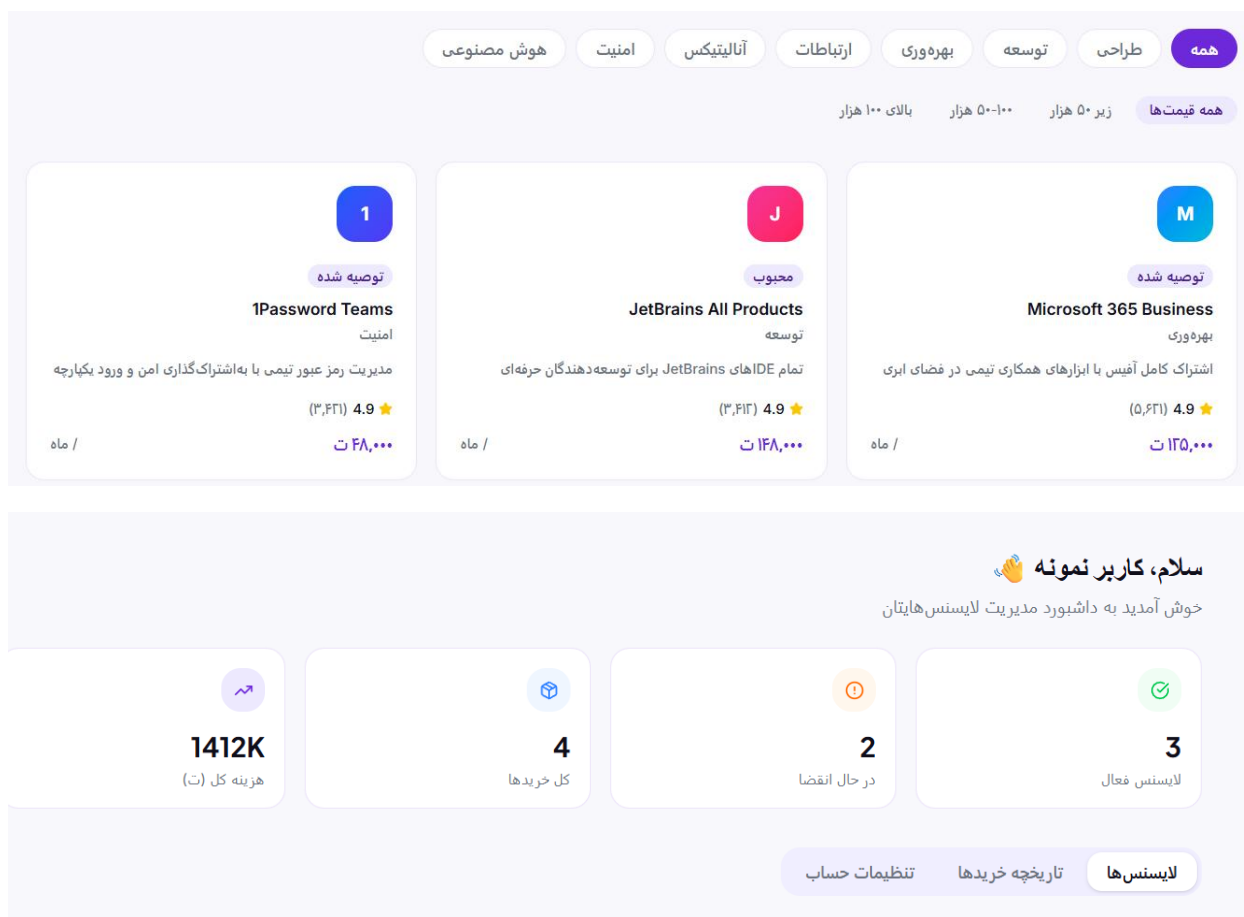


شکل ۵ -- دیگرام پایگاه‌داده (Database Domain Model)

همچنین در همین سطح، دو دیگرام تکمیلی برای پوشش دقیق‌تر مازول فروشنده و ارتباط میان فایل‌های پروژه ترسیم شد که در فاز سوم و پس از تکمیل کد نهایی، این دو دیگرام بازبینی و به‌روزرسانی شدند (شرح کامل در بخش ۳.۲):

۲.۲ نمونه‌ی اولیه در Figma

یک نمونه‌ی اولیه (پروتوتایپ) از نرم‌افزار با استفاده از ابزار Figma طراحی شد. در طراحی این پروتوتایپ، صرفاً به صفحات ایستا بسنده نشد؛ بلکه جریان‌های کاربری، انشعاب‌های منطقی و رفتارهای سیستم در سناریوهای مختلف (از جمله مدیریت خطاها و حالت‌های موفقیت) نیز در قالب این پروتوتایپ شبیه‌سازی شد تا مبنای اصلی پیاده‌سازی نهایی قرار گیرد.



شکل ۸ و ۹ -- نمونه‌هایی از پروتوتایپ طراحی‌شده در Figma

۲.۳ آغاز توسعه‌ی پروژه

برای شروع پیاده‌سازی، یک مخزن خصوصی در GitHub برای تیم ایجاد شد (SBU-SWE-Project). در ادامه، مدل داده‌ی (Data Model) پروژه با استفاده از Prisma تعریف و پایگاه‌داده‌ی PostgreSQL راه‌اندازی شد. فیچرهای پایه‌ای مانند احراز هویت کاربران (ثبت‌نام و ورود مبتنی بر JWT) و تعیین سطوح دسترسی کاربران (نقش‌های BUYER، SELLER و ADMIN) نیز در همین فاز پیاده‌سازی شدند و پایه‌ی توسعه‌ی فیچرهای اصلی در فاز سوم را فراهم کردند.

۳. فاز سوم پروژه: توسعه، تست و استقرار

در این فاز، پنج نیازمندی عملکردی اصلی پروژه بر اساس سند SRS به طور کامل پیاده سازی، تست و مستقر شدند. در ادامه، فعالیت های انجام شده در این فاز به تفکیک شش بخش خواسته شده در صورت پروژه شرح داده می شود.

۳.۱ تطابق فرآیند توسعه با نیازمندی های سند SRS

پنج نیازمندی عملکردی اصلی که در سند SRS در فاز اول تعریف شده بودند، در این فاز به طور کامل پیاده سازی و با تست واحد پوشش داده شدند. جدول زیر تطابق هر نیازمندی را با ماژول پیاده سازی شده و فایل تست مربوطه نشان می دهد:

ردیف	نیازمندی عملکردی (SRS)	ماژول / فایل پیاده سازی	تست واحد مرتبط
۱	احراز هویت کاربران (ثبت نام، ورود، مدیریت نقش ها با JWT)	authController.js authMiddleware.js (verifyToken)	auth.test.js
۲	مدیریت محصولات توسط فروشنده (ثبت محصول جدید، ویرایش با کنترل مالکیت)	sellerController.js sellerRoutes.js (isSeller)	seller.test.js product.test.js
۳	مشاهده و خرید محصولات، تسویه حساب سبد خرید توسط خریدار	orderController.js buyerController.js	order.test.js, buyer.test.js
۴	تأیید/رد محصولات و مدیریت کاربران (بن، تغییر نقش) توسط ادمین	adminController.js	admin.test.js
۵	مدیریت اشتراک ها (Subscription) و دسترسی خریداران به محتوای اشتراکی	subscriptionController.js	subscription.test.js

همان طور که در جدول بالا مشخص است، هر نیازمندی عملکردی به طور مستقیم به یک یا چند کنترلر در بک اند و حداقل یک فایل تست واحد نگاشت می شود؛ به این ترتیب فرآیند توسعه ی پروژه کاملاً منطبق با نیازمندی های مستند شده در سند SRS پیش رفته است.

۳.۲ تکمیل سند معماری با مدل C۴ (تا سطح کد)

سند معماری که در فاز دوم با مدل C۴ تا سطح ۳ (کامپوننت‌ها) طراحی شده بود، در این فاز تکمیل شد و دیاگرام‌های سطح ۳ برای انطباق کامل با کد نهایی و ساختار واقعی سیستم بازبینی شدند. برای این کار، دو نوع دیاگرام تکمیلی با PlantUML تهیه شد: یکی دیاگرام Component برای نمایش اجزای اصلی ماژول Seller و دیگری دیاگرام Source File برای نمایش رابطه‌ی دقیق میان فایل‌های پروژه. در ادامه، فرآیند تکمیل این مستندسازی برای ماژول فروشنده به تفصیل شرح داده می‌شود.

در بخش فرانت‌اند، مسیر اصلی مرتبط با فروشنده در App.jsx تعریف می‌شود و کاربر پس از ورود به بخش فروشنده، به صفحه‌ی SellerDashboard.jsx هدایت می‌شود. این فایل در واقع مهم‌ترین نقطه‌ی فرانت‌اند در ماژول فروشنده است، زیرا هم داده‌های مربوط به فروشنده را از بک‌اند دریافت می‌کند و هم مسئول نمایش آن‌ها در قالبی قابل فهم و کاربردی است. در این داشبورد، اطلاعاتی مانند لیست محصولات فروشنده، مشتریان مرتبط، آمار کلی فروش و همچنین نظرات ثبت شده برای محصولات نمایش داده می‌شود. برای نظم بهتر در طراحی رابط کاربری، اجزای مختلف صفحه به کامپوننت‌های کوچک‌تر تفکیک شده‌اند. برای مثال Navbar.jsx برای نمایش نوار ناوبری، CreateProductModal.jsx برای ثبت محصول جدید، EditProductModal.jsx برای ویرایش اطلاعات محصول، SellerProducts.jsx برای نمایش لیست محصولات، SellerCustomers.jsx برای نمایش فهرست مشتریان، و SharedCustomTable.jsx برای نمایش جدولی داده‌ها استفاده شده‌اند.

نکته‌ی مهم در طراحی فرانت‌اند این است که داشبورد فروشنده تنها یک صفحه‌ی نمایشی نیست، بلکه نقش هماهنگ‌کننده بین اجزای مختلف را نیز بر عهده دارد. زمانی که فروشنده قصد ایجاد محصول جدید را دارد، فرم مربوطه در CreateProductModal.jsx تکمیل می‌شود، اما عملیات واقعی ذخیره‌سازی از طریق SellerDashboard.jsx و با ارسال درخواست HTTP به بک‌اند انجام می‌گیرد. همین روند برای ویرایش محصول نیز وجود دارد؛ به این صورت که EditProductModal.jsx صرفاً رابط ورود اطلاعات را فراهم می‌کند و عملیات نهایی از طریق داشبورد و با فراخوانی API مربوطه به سرور ارسال می‌شود. این ساختار نشان می‌دهد که طراحی پروژه تنها بر پایه‌ی ظاهر نبوده، بلکه اصول مدیریت داده و جداسازی مسئولیت‌ها نیز رعایت شده است.

در سمت بک‌اند، نقطه‌ی آغاز کار server.js است. این فایل سرور Express را راه‌اندازی می‌کند و مسیرهای مرتبط با بخش فروشنده را از طریق sellerRoutes.js روی پیشوند /api/seller mount می‌کند. همین موضوع باعث می‌شود که هرچند در فایل روت‌ها مسیرهایی مانند /product، /analytics، و /product-edit تعریف شده‌اند، اما در عمل مسیرهای نهایی و قابل فراخوانی به ترتیب /api/seller/product، /api/seller/analytics، و /api/seller/product-edit باشند. این مسئله در تحلیل پروژه و رسم نمودارها اهمیت زیادی داشت، چون یکی از اشتباهات رایج این است که مسیرهای تعریف شده در فایل Route با مسیرهای واقعی قابل دسترس در سرور یکسان فرض شوند، در حالی که mount شدن آن‌ها در server.js ساختار نهایی API را مشخص می‌کند.

در فایل `sellerRoutes.js` مسیرهای اصلی فروشنده تعریف شده‌اند و برای همه‌ی این مسیرها از دو `middleware` کلیدی استفاده می‌شود: `verifyToken` و `isSeller`. `middleware` اول وظیفه دارد اعتبار توکن کاربر را بررسی کند و در صورت معتبر بودن، اطلاعات او را در اختیار ادامه‌ی فرایند قرار دهد. اما نکته‌ی مهم‌تر این است که این `middleware` صرفاً به بررسی توکن اکتفا نمی‌کند، بلکه از طریق `Prisma` به پایگاه داده نیز متصل می‌شود تا وضعیت کاربر را بررسی کند؛ برای مثال مشخص شود آیا کاربر بن شده است یا خیر. این وابستگی مستقیم به `Prisma` در تحلیل پروژه اهمیت زیادی داشت، چون نشان می‌دهد احراز هویت در این سیستم تنها یک بررسی سطحی سمت سرور نیست، بلکه با داده‌های واقعی ذخیره‌شده در پایگاه داده کنترل می‌شود. `middleware` دوم یعنی `isSeller` نقش کاربر را بررسی می‌کند تا فقط کاربرانی که واقعاً نقش `SELLER` دارند بتوانند از مسیرهای این بخش استفاده کنند.

منطق اصلی پروژه در `sellerController.js` قرار دارد. این فایل شامل سه تابع اصلی است که هر کدام بخشی از نیازهای فروشنده را پوشش می‌دهند. تابع `sellerCreateProduct()` مسئول ثبت محصول جدید است و اطلاعات دریافت‌شده از فرانت‌اند را با استفاده از `Prisma` در جدول محصولات ذخیره می‌کند. تابع `updateSellerProduct()` وظیفه‌ی ویرایش محصولات موجود را بر عهده دارد، اما قبل از ویرایش تنها به پیدا کردن محصول اکتفا نمی‌کند؛ بلکه بررسی می‌کند که آیا محصول موردنظر واقعاً متعلق به همان فروشنده‌ای هست که درخواست ویرایش را ارسال کرده یا نه. این کنترل مالکیت یکی از مهم‌ترین بخش‌های امنیتی سیستم است، چون مانع از آن می‌شود که یک فروشنده بتواند محصول فروشنده‌ی دیگری را تغییر دهد. علاوه بر این، در منطق پروژه پس از ویرایش محصول، وضعیت آن دوباره به `PENDING` تغییر می‌کند تا فرایند بررسی یا تأیید مجدد روی آن انجام شود. این بخش نشان می‌دهد که سیستم فقط به تغییر داده بسنده نکرده و قواعد تجاری مرتبط با مدیریت محصولات را نیز در نظر گرفته است. تابع سوم یعنی `getSellerAnalytics()` وظیفه‌ی تولید داده‌های تحلیلی برای داشبورد فروشنده را بر عهده دارد. این تابع با استفاده از روابط میان مدل‌های مختلف پایگاه داده، اطلاعاتی از قبیل محصولات فروشنده، سفارش‌های مرتبط، مشتریان، و نظرات ثبت‌شده را استخراج می‌کند و به شکلی مناسب برای نمایش در داشبورد در اختیار فرانت‌اند قرار می‌دهد.

در لایه‌ی داده، استفاده از `Prisma` و `PostgreSQL` یکی از نقاط قوت معماری این پروژه است. در این ساختار، داده‌های اصلی در مدل‌هایی مانند `User`، `Product`، `Order`، `OrderItem` و `Review` ذخیره می‌شوند. یکی از نکات مهمی که در بررسی پروژه مشخص شد این بود که «فروشنده» در این سیستم یک مدل جداگانه در پایگاه داده ندارد، بلکه در واقع یک `User` است که مقدار فیلد `role` او برابر `SELLER` قرار گرفته است. این نکته از آن جهت اهمیت دارد که در ابتدا ممکن است تصور شود `Seller` باید موجودیت مستقلی باشد، اما در طراحی واقعی پروژه، نقش‌ها از طریق یک مدل مشترک مدیریت شده‌اند. محصولات از طریق `sellerId` به کاربر فروشنده متصل می‌شوند، سفارش‌ها شامل آیتم‌هایی هستند که به محصولات اشاره دارند، و نظرات نیز به محصولات و کاربران مربوط می‌شوند. همین روابط باعث می‌شود که در تابع تحلیل فروشنده بتوان اطلاعات مرتبط با فروش، مشتریان و نظرات را به صورت یکپارچه بازیابی کرد.

بخش مهم دیگری از این کار، مستندسازی تصویری پروژه با استفاده از `PlantUML` بود. اهمیت این نمودارها در آن بود که فقط یک نمایش کلی و ظاهری ارائه نکنند، بلکه دقیقاً با ساختار واقعی کد و وابستگی‌های پروژه منطبق باشند. برای مثال در بازبینی

نمودارها مشخص شد که لازم است وابستگی authMiddleware.js به Prisma و در نهایت پایگاه داده نیز نمایش داده شود، چون این فایل فقط یک middleware ساده نیست و در عمل برای بررسی وضعیت کاربر به بانک اطلاعاتی متکی است. همچنین لازم بود در نمودارها به صورت شفاف نشان داده شود که SellerDashboard.jsx درخواست‌های GET /api/seller/product، POST /api/seller/product/edit و PUT /api/seller/product را به سرور ارسال می‌کند و داده‌های بازگشتی را در اجزای مختلف رابط کاربری توزیع می‌کند. نکته‌ی دیگر این بود که تابع‌های getSellerAnalytics، sellerCreateProduct() و updateSellerProduct() باید به عنوان بخش داخلی sellerController.js نمایش داده می‌شدند تا از نظر source-file diagram، ساختار واقعی پروژه حفظ شود.

یکی از جنبه‌های قابل توجه در این پروژه، دقت در اصلاح جزئیات برای رسیدن به یک مستند کاملاً منطبق با کد بود. در طول بررسی‌ها مشخص شد که برخی نمایش‌های اولیه اگرچه از نظر مفهومی درست بودند، اما هنوز با واقعیت پروژه فاصله داشتند. برای نمونه، لازم بود در نمودارها تفاوت میان route تعریف شده در فایل sellerRoutes.js و route نهایی mount شده در server.js مشخص شود. همچنین در مسیر ویرایش محصول، کنترل مالکیت و بازنشانی وضعیت محصول به PENDING جزو بخش‌هایی بودند که اگر در مستندات دیده نمی‌شدند، تصویر ناقصی از رفتار واقعی سیستم ارائه می‌دادند. این اصلاحات نشان می‌دهد که هدف این کار تنها تهیه‌ی یک نمودار زیبا نبود، بلکه دستیابی به مستندی بود که بتوان به آن به عنوان بازتاب دقیق ساختار پروژه اعتماد کرد.

دیagramهای نهایی سطح ۳ برای ماژول Seller و ارتباط میان فایل‌های پروژه، که پس از این بازبینی و انطباق با کد نهایی به‌روزرسانی شدند، در شکل‌های ۶ و ۷ (بخش ۲.۱) ارائه شده‌اند. توجه شود که سطح ۴ (کد) مطابق صورت پروژه در فاز دوم مورد نیاز نبود و در همین فاز سوم، در قالب diagramهای Component و Source File توسط PlantUML تکمیل شد.

۳.۳ نوشتن Unit Test برای APIs سمت بک‌اند

برای پوشش نیازمندی‌های عملکردی اصلی پروژه، تست‌های واحد با استفاده از فریم‌ورک Jest و کتابخانه‌ی Supertest برای فراخوانی APIهای Express نوشته شدند. در مجموع ۸ فایل تست شامل بیش از ۳۰۱ سناریوی تست جداگانه، برای ۸ گروه از سرویس‌های بک‌اند تهیه شد که به مراتب بیش از حداقل سه API خواسته شده در صورت پروژه را پوشش می‌دهد. در هر فایل تست، وابستگی‌های خارجی مانند Prisma Client و jsonwebtoken با استفاده از jest.mock به صورت ایزوله شبیه‌سازی شده‌اند تا تست‌ها بدون نیاز به اتصال واقعی به پایگاه داده اجرا شوند. نام هر سناریوی تست (در بلوک‌های it) به گونه‌ای انتخاب شده که به وضوح بیانگر رفتار مورد انتظار نیازمندی مرتبط باشد؛ برای مثال «should block update if seller does not own the product» مستقیماً به نیازمندی کنترل مالکیت محصول در فاز سوم اشاره دارد. جدول زیر خلاصه‌ای از فایل‌های تست و سناریوهای پوشش داده شده را نشان می‌دهد:

فایل تست	سناریوهای پوشش داده شده	API مورد تست
auth.test.js	ثبت نام موفق، خطای ایمیل تکراری، ورود موفق، مسدودسازی کاربر بن شده	POST /api/auth/register, POST /api/auth/login
seller.test.js	دریافت تحلیل فروشنده، ویرایش محصول توسط مالک، رد ویرایش برای غیرمالک	GET /api/seller/analytics, PUT /api/seller/product-edit
product.test.js	دریافت لیست محصولات عمومی، ثبت محصول توسط فروشنده، رد ثبت توسط خریدار	GET /api/products, POST /api/seller/product
order.test.js	تسویه حساب موفق سبد خرید، خطای سبد خرید خالی	POST /api/orders/checkout
buyer.test.js	دریافت کتابخانه خریدهای خریدار، بررسی دسترسی اشتراکی	GET /api/buyer/dashboard, GET /api/buyer/digicourse/:productId
admin.test.js	تأیید وضعیت محصول توسط ادمین، رد دسترسی خریدار به مسیرهای ادمین	POST /api/admin/verify-product
subscription.test.js	دریافت پلن های فعال اشتراک، وضعیت اشتراک کاربر	GET /api/subscriptions/plans, GET /api/subscriptions/status
user.test.js	ویرایش پروفایل، تغییر رمز عبور صحیح/نادرست	PUT /api/user/profile, PUT /api/user/password

اجرای مجموعه‌ی تست‌ها با دستور npm test (که در پس زمینه دستور jest را در پوشه‌ی backend اجرا می‌کند) انجام می‌شود و همان دستوری است که در پایپ لاین CI پروژه نیز فراخوانی می‌شود.

۳.۴ داکرایز کردن پروژه (Dockerize)

کل پروژه شامل پایگاه داده، کد سمت سرور (Back-end)، کد سمت کلاینت (Front-end) و سرویس‌های وابسته، با استفاده از Docker و Docker Compose به گونه‌ای پیکربندی شده‌اند که با دستورات `docker build` و `docker compose up` به راحتی ساخته و اجرا شوند. ساختار سرویس‌ها در فایل `docker-compose.yml` به شرح زیر است:

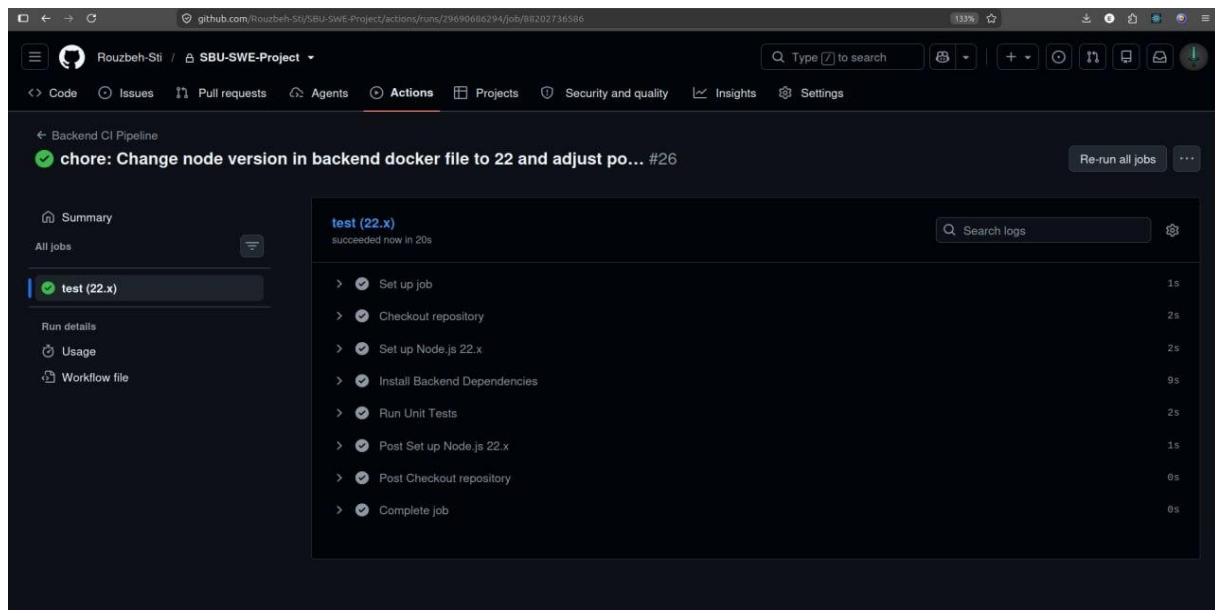
- سرویس `postgres-db`: بر پایه‌ی `postgres:۱۵-alpine` رسمی، همراه با `healthcheck` برای اطمینان از آماده بودن پایگاه داده پیش از اجرای بک‌اند، و یک `volume` با نام `postgres_data` برای ماندگاری داده‌ها.
- سرویس `backend-app`: بر پایه‌ی `Dockerfile` اختصاصی پوشه‌ی `backend (node:۲۲)`؛ ابتدا وابستگی‌ها نصب و `Prisma Client` تولید می‌شود، سپس پیش از اجرای `npm start`، دستور `npx prisma migrate deploy` برای اعمال خودکار مایگریشن‌های پایگاه داده اجرا می‌شود. این سرویس با استفاده از `depends_on` و شرط `service_healthy`، تنها پس از سالم بودن سرویس پایگاه داده شروع به کار می‌کند.
- سرویس `frontend-app`: بر پایه‌ی `Dockerfile` اختصاصی پوشه‌ی `frontend (node:۲۰-alpine)`، که پس از نصب وابستگی‌ها، سرور توسعه‌ی `Vite` را روی پورت ۵۱۷۳ اجرا می‌کند و آدرس بک‌اند را از طریق متغیر محیطی `VITE_API_URL` دریافت می‌کند.

تمامی متغیرهای حساس مانند `JWT_SECRET` و مشخصات اتصال به پایگاه داده از طریق متغیرهای محیطی (با مقادیر پیش فرض امن برای محیط توسعه) به کانتینرها تزریق می‌شوند، به گونه‌ای که کل پروژه تنها با اجرای یک دستور `docker compose up --build` از حالت کد منبع به یک سیستم کاملاً در حال اجرا (پایگاه داده + بک‌اند + فرانت‌اند) تبدیل می‌شود.

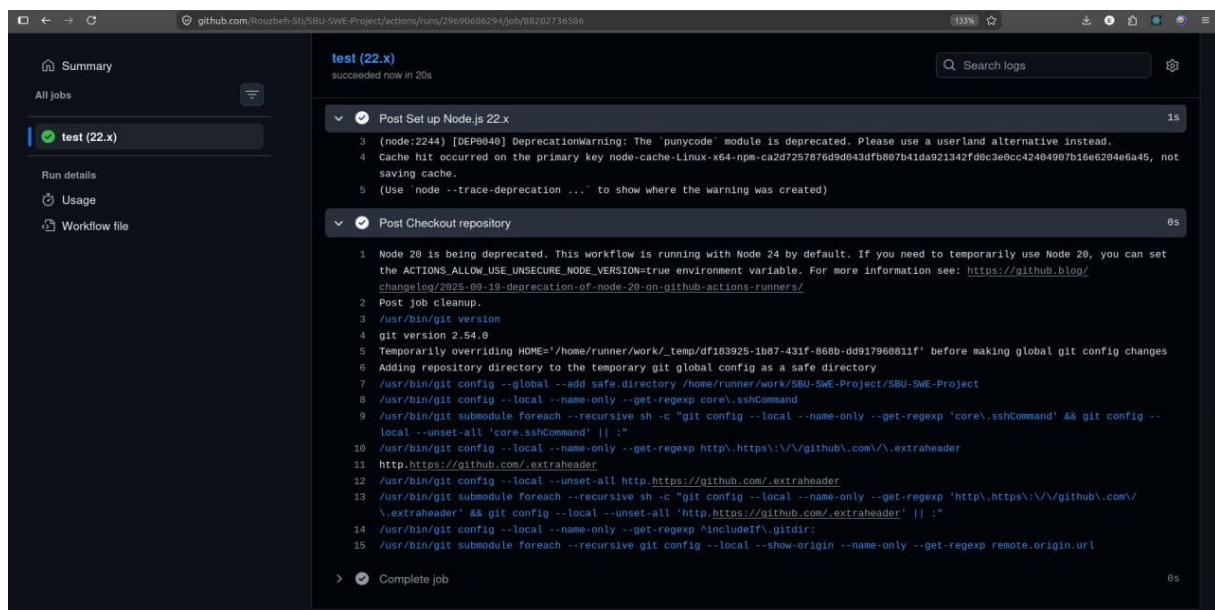
۳.۵ پیاده‌سازی خودکارسازی (CI) پیش از ادغام به شاخه‌ی اصلی

فرآیند یکپارچه‌سازی مداوم (CI) با استفاده از `GitHub Actions` پیاده‌سازی و در مخزن پروژه، در مسیر `github/workflows/ci.yml`، تعریف شده است. این پایپ‌لاین (با نام `Backend CI Pipeline`) به گونه‌ای پیکربندی شده که پیش از هر `merge` به شاخه‌های اصلی `main` یا `master` (چه از طریق `push` مستقیم و چه از طریق `pull request`)، به طور خودکار اجرا می‌شود. مراحل اجرای این پایپ‌لاین به ترتیب عبارت‌اند از: واکنشی کد مخزن (`Checkout`)، برپاسازی محیط `Node.js` نسخه‌ی ۲۲، نصب وابستگی‌های بک‌اند با `npm install`، و در نهایت اجرای کامل مجموعه‌ی تست‌های واحد با دستور `npm test` در پوشه‌ی `backend`؛ برای اجرای تست‌ها نیز متغیرهای محیطی مورد نیاز مانند `JWT_SECRET` به صورت مقادیر آزمایشی (`dummy`) در اختیار پایپ‌لاین قرار می‌گیرند. تنها در صورت موفقیت تمامی تست‌ها، وضعیت `CI` روی `commit` یا `pull request` سبز می‌شود و امکان `merge` به شاخه‌ی اصلی فراهم می‌گردد؛ در غیر این صورت `GitHub Actions` مانع ادغام کد ناقص یا خراب به شاخه‌ی اصلی می‌شود.

در ادامه، گزارش اجرای موفق این پایپ‌لاین در مخزن پروژه (`Rouzbeh-Sti/SBU-SWE-Project`) ارائه شده است:



شکل ۱۰ -- نمای کلی اجرای موفق پایپ‌لاین CI روی «commit chore: Change node version in backend docker file to ۲۲ and adjust po...» (تمامی مراحل با علامت سبز تکمیل شده‌اند)



شکل ۱۱ -- جزئیات گام‌های 'Post Set up Node.js' و 'Post Checkout repository' در لاگ اجرای CI، که موفقیت کامل job با نام 'test (۲۲.x)' را تأیید می‌کند

۳.۶ ریسپانسیو کردن سمت فرانت‌اند (Responsive)

ظاهر پروژه در سمت فرانت‌اند با استفاده از کلاس‌های واکنش‌گرای Tailwind CSS (پیشوندهایی مانند lg، md، sm، و xl) برای صفحه‌نمایش‌های متداول (موبایل، تبلت و دسکتاپ) بهینه‌سازی شده است. این رویکرد در بیش از ۱۵ کامپوننت و صفحه‌ی اصلی پروژه از جمله Navbar، Hero، Footer، Marketplace، BuyerDashboard، SellerDashboard، AdminDashboard، Cart، ProductDetails، Home، Settings، AuthLayout و EditProductModal و CreateProductModal به کار رفته است تا اجزا، چیدمان و ناوبری متناسب با اندازه‌ی صفحه تغییر کنند؛ علاوه بر این، در فایل استایل کلی پروژه (App.css) نیز از media queryهای استاندارد CSS برای تنظیمات تکمیلی در نقاط شکست (breakpoints) مختلف استفاده شده است. طراحی رابط کاربری داشبوردها (خریدار، فروشنده و ادمین) به گونه‌ای صورت گرفته که جدول‌ها و کارت‌های اطلاعاتی در صفحه‌نمایش‌های کوچک به صورت تک‌ستونی و در صفحه‌نمایش‌های بزرگ‌تر به صورت چندستونی نمایش داده شوند.

۴. نتیجه‌گیری کلی

در مجموع، این پروژه نمونه‌ای از یک چرخه‌ی کامل مهندسی نرم‌افزار است که از شناخت مسئله و استخراج نیازمندی‌ها (فاز اول)، عبور از طراحی معماری و نمونه‌سازی اولیه (فاز دوم)، تا توسعه‌ی کامل، تست، داکرایز کردن و استقرار خودکار (فاز سوم) را در بر می‌گیرد. سامانه‌ی نهایی، یک بستر خرید و فروش دیجیتال با معماری کاملاً مجزای Front-end و Back-end است که در آن هم تجربه‌ی کاربری در فرانت‌اند مورد توجه قرار گرفته و هم امنیت، کنترل دسترسی، مالکیت داده، و منطق تجاری در بک‌اند به درستی پیاده‌سازی شده است. استفاده از React در رابط کاربری، Express در لایه‌ی سرور، Prisma به عنوان ORM و PostgreSQL به عنوان پایگاه‌داده، ترکیبی مناسب و استاندارد برای چنین سامانه‌ای ایجاد کرده است. از طرف دیگر، مستندسازی دقیق معماری با مدل C۴ و نمودارهای PlantUML، پوشش تست واحد گسترده با Jest و Supertest، داکرایز کامل پروژه، و راه‌اندازی پایپ‌لاین CI با GitHub Actions باعث شده ساختار پروژه نه تنها از دید توسعه‌دهنده، بلکه از دید هر ارزیاب یا اعضای جدید تیم نیز قابل اعتماد و قابل نگهداری باشد.